

Guide de Déploiement — Night Sky Viewer

Architecture de déploiement

Service	Plateforme	URL
Backend (FastAPI)	Render	<code>https://<ton-service>.onrender.com</code>
Frontend (React/Vite)	Vercel	<code>https://projet-poo.vercel.app</code>

⚠ **Render free tier** : le service se met en veille après ~15 min d'inactivité. La première requête après une période d'inactivité peut prendre 30–60 sec (cold start). Pour éviter ça, passer sur un plan payant ou utiliser un cron de ping.

1. Vérifier l'état d'un déploiement

Render (backend)

1. Dashboard render.com → ton service
2. Onglet **"Events"** : timeline des déploiements avec statut `Live` ✅ ou `Failed` ❌
3. Onglet **"Logs"** : logs en temps réel (erreurs Python au démarrage visibles ici)
4. Health check direct : `https://<ton-service>.onrender.com/api/health`
 - Réponse attendue : `{"status": "healthy", "database": "connected", "star_count": 119614}`

Vercel (frontend)

1. Dashboard vercel.com → ton projet → onglet **"Deployments"**
2. Le dernier commit doit apparaître avec le statut **"Ready"** ✅
3. Vercel envoie aussi un email en cas d'échec de build

2. Forcer un redéploiement manuel

Render

- Dashboard → service → bouton **"Manual Deploy"** → **"Deploy latest commit"**

Vercel

- Dashboard → Deployments → dernier déploiement → **"Redeploy"**

Via git (les deux à la fois)

```
git commit --allow-empty -m "chore: trigger deploy"
git push origin main
```

3. Redéployer après des changements de code

Le déploiement est **automatique** dès qu'un commit est poussé sur la branche `main`.

Workflow standard

```
# 1. Faire tes modifications sur la branche v2
git checkout v2

# ... modifier les fichiers ...

# 2. Committer sur v2
git add <fichiers modifiés>
git commit -m "feat/fix: description du changement"

# 3. Merger v2 dans main
git checkout main
git merge v2 --no-ff -m "Merge V2 : description"

# 4. Pousser → Render et Vercel redéploient automatiquement
git push origin main

# 5. Revenir sur v2 pour la suite du développement
git checkout v2
```

Render redéploie en ~2-3 min. Vercel redéploie en ~30-60 sec.

4. Variables d'environnement

Render (backend)

À configurer dans : **Dashboard** → **Service** → **Environment**

```
ENVIRONMENT=production
DEBUG=false
DATABASE_URL=sqlite:///./data/nightsky.db # ou PostgreSQL si utilisé
CORS_ORIGINS=["https://projet-poo.vercel.app"]
CACHE_TTL_STARS=600
CACHE_MAX_SIZE=512
DEFAULT_MAG_LIMIT=6.0
MAX_STARS_RESPONSE=10000
```

⚠️ Après chaque modification de variable, Render redémarre le service automatiquement.

Vercel (frontend)

À configurer dans : **Dashboard** → **Project** → **Settings** → **Environment Variables**

```
VITE_API_URL=https://<ton-service>.onrender.com
```

⚠️ Après modification d'une variable Vercel, il faut **redéployer manuellement** : Deployments → sur le dernier déploiement → **Redeploy**

5. Arrêter / Suspendre le service

Render

- **Suspendre** (gratuit, sans supprimer) : Dashboard → service → **Settings** → **Suspend Service**
 - **Reprendre** : même endroit → **Resume Service**
-

6. Dépannage rapide

Symptôme	Cause probable	Solution
CORS error dans la console	URL Vercel absente de <code>CORS_ORIGINS</code>	Mettre à jour la var Render
API unreachable	Mauvaise <code>VITE_API_URL</code> sur Vercel	Corriger + Redeploy Vercel
Réponse lente (30–60 sec)	Cold start Render free tier	Normal après inactivité ; passer sur plan payant
<code>star_count: 0</code>	Base SQLite absente ou vide	Vérifier que <code>data/nightsky.db</code> est bien dans le repo
Logs Render : crashes répétés	Erreur Python au démarrage	Voir Render → Logs
Build Vercel échoue	Erreur TypeScript ou dépendance manquante	Voir Vercel → Deployments → Build Logs